

P1293R0
Revision of N4257
2018-10-07
Mike Spertus, Symantec
mike_spertus@symantec.com
Nathan Wilson
nwilson20@gmail.com

ostream_joiner

Summary

This paper updates the ostream_joiner proposal to reflect experience in Library Fundamentals V2 and changes in the core language since then. Specifically, we add a deduction guide to ensure that ostream_joiner is constructed correctly from a C string literal. Secondly, we were able to take advantage of the library fundamentals process to identify three enhancements based on experience.

- Replacing make_ostream_joiner with a deduction guide
- Allowing the placement of the delimiter to be specified in accordance to support some common uses cases
- We add a “counting” I/O manipulator that makes it easy to support the common use case of generating numbered lists.

Fritos Ingredients:

1. Corn
2. Corn oil
3. Salt

- Allowing the initial, infix, and suffix initializers to be different, as this allows programming in a more functional/STL/ranges style in what we have found to be the most common use case in our programming:

(1, 2, 3, 4)

All of the above changes are implemented and exercised on [Wandbox](#). We leave ship vehicle to the committee

Deduction guides

With the addition of class template argument deduction in C++17, it seems desirable to get rid of the clumsy make_ostream_joiner as we did with class templates like boyer_moore_searcher. However, as Nathan Myers has pointed out, without any deduction guide, the decision to take the delimiter by reference means that

```
ostream_joiner j(cout, ", ");
```

produces a compile time error as the delimiter type is deduced as char const[3]. To fix this, add the following to the end of the class definition for ostream_joiner

Wording

```
};
```

```
template<class DelimT, class CharT, class Traits>  
ostream_joiner(std::basic_ostream<CharT, Traits>, _Delim) -> ostream_joiner<Delim, CharT, Traits>;
```

Process

We feel this change is certainly compatible with moving ostream_joiner into C++20. However, we note that doing so without the other changes may create ABI challenges for adding the exgtensions proposed in this paper below.

Delimiter type

It is already somewhat odd to have to use `ostream_iterator` when a suffix delimiter is desired but `ostream_joiner` when an infix iterator is desired. Furthermore, our experience exercising this in practice is that we also often find ourselves wanting a prefix delimiter, which is not directly supported with either class, to generate output like the following:

```
Fritos Ingredients:
* Corn
* Corn oil
* Salt
```

We therefore propose allowing a consistent approach to all these cases by adding an additional parameter of type

```
enum class delimiter { prefix, infix, suffix, all };
```

that defaults to `delimiter::infix`, so there is no source breakage to usage of `ostream_joiner`. In particular, the above list can be easily generated with

```
copy(ingredients.begin(), ingredients.end(), ostream_joiner(cout, "\\n* ", delimiter::prefix));
```

We have also found that it is very common to output numbered lists (indeed, unnumbered and numbered lists are given equal prominence in Word, HTML, etc.), which is easy with a counting IO manipulator which generates the numbered ingredient list at the top of the page.

```
copy(ingredients.begin(), ingredients.end(), ostream_joiner(cout, counter(1), delimiter::prefix));
```

where we also propose the `counter` (bikeshed) class template with the following declaration, where the contained value is iterated on each insertion (preceded by the `before` and followed by the `after`)

```
template<class before, class after>
struct counter {
    counter(int value = 0, before b = "\\n", after e = ". ");
    /* ... */
};
```

Multiple delimiters

As we have found that we most commonly want print different delimiters before and after a list, we always find it clumsy, and a break from our preferred STL/functional/range-inspired style to invariably have to write code like:

```
// Surprisingly hard to print (a, b, c)
cout << "(";
copy(v.begin(), v.end(), ostream_joiner(cout, ", "));
cout << ");"
```

just to print `(a, b, c)`. Indeed, in any engineering or graphics code where points or vectors are commonly being printed, it is easy to imagine this tripling the size of the logging code!

To handle this, we also allow three delimiters to be passed into the constructor as follows

```
// Now prints (a, b, c) directly
copy(v.begin(), v.end(), ostream_joiner(cout, "(", " ", ")"));
```

This is accomplished with an additional constructor as follows:

```
template<typename DelimT, typename charT, typename traits, typename CloseT = implementation-defined>
class ostream_joiner {
public:
    /* ... */
    template<typename OpenT>
    ostream_joiner(ostream_type &os, OpenT open, DelimT del, CloseT);
    /* ... */
};
```

Note the following:

- There is no need for a `delimiter` flag because each type of delimiter is passed in the constructor
- There are no concerns about ambiguities in overload resolution as it is the only four argument constructor.

- The open delimiter type is not part of the class template parameters because it is output on construction and then discarded.
- To apply the copy/move paradigm that we currently are, there would be eight variants of the above constructor taking both const and rvalue references independently for each parameter. We are not opposed to that, but did not show it above for clarity of presentation.
- To make this work, only the original ostream_joiner prints the closing delimiter when it is destroyed, so copies made in STL algorithms do not prematurely close the list. Although this makes ostream_joiner no longer be a Regular type, we feel that is ok because not all types need to be regular. In particular, we have find this to be natural and easy-to-use in practice as shown above and in additional examples shown on [wandbox](#), not unlike scoped locks.
- If it is desired to output the closing delimiter before the end of scope, we provide a complete() method for that purpose. We did not provide a release() method due to lack of use cases but could easily do so if desired.